UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Most of the slides are taken from

Eric Freeman, Elisabeth Robson, Bert Bates, Kathy SierraHead First Design Patterns: Building Extensible and Maintainable Object-Oriented Software, O'Reilly Media 2020

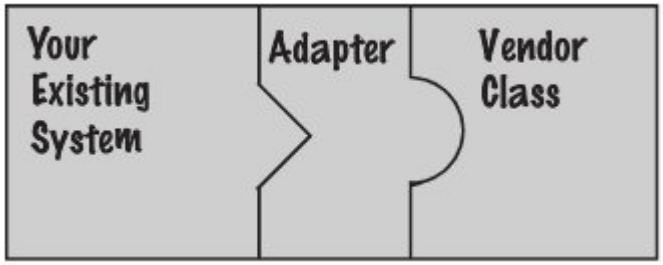
Design Pattern 4
Adapter, Facade and Template

Francesco Guerra
francesco.guerra@unimore.it

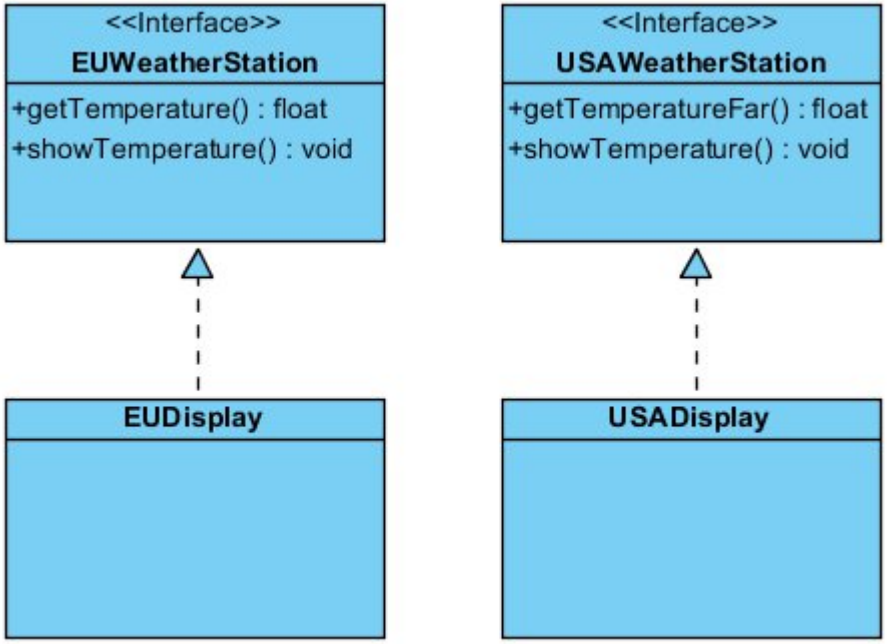
The Adapter Pattern

- We can adapt a design expecting one interface to a class that implements a different interface
- Let us consider an existing software system. You need to work a new vendor class library into, but the new vendor designed their interfaces differently than the last vendor

- The adapter acts as the middleman by receiving requests from the client and converting them into requests that make sense on the vendor classes.



Example



The idea is to make the EUDisplay able to show the temperature expressed by a Station measuring in Fahrenheit

Example

```
public interface EUWeatherStation{
    public float getTemperature();
    public void showTemperature();
}
```

```
public interface USAWeatherStation{
    public float getTemperatureFar();
    public void showTemperature();
}
```

```
public class EUDisplay implements
EUWeatherStation{
    public float getTemperature() {
    System.out.println("Temperature
in Celsius");
    return 22.7; //example
    }

    public void showTemperature() {
        System.out.println("This is
the temperature in Celsius");
    }
}
```

```
public class USADisplay implements
UsaWeatherStation{
    public float getTemperatureFar()
{
     System.out.println("Temperature
in Fahrenheit");
          return 76; //example
    }

    public void showTemperature() {
        System.out.println("This is
the temperature in Fahrenheit");
    }
}
```

The adapter requires a class of the adaptee

Example: the Adapter

```
public class EUUSADisplayAdapter implements EUWeatherStation {
    USADisplay newStation;
    public EUUSADisplayAdapter(USADisplay newStation) {
        this.newStation = newStation;
    }
    public float getTemperature() {
        return ((newStation.getTemperatureFar()-32)*5/9);
    }
    public void showTemperature() {
         System.out.println("This is the temperature " +
(newStation.getTemperatureFar()-32)*5/9);    }
}

public class TestDrive {
    public static void main(String[] args) {
         EUDisplay euDisplay= new EUDisplay();
    USADisplay usaDisplay= new USADisplay();
    EUWeatherStation wStation= new EUUSADisplayAdapter(usaDisplay);

         test(wStation);
    }
    static void test(EUWeatherStation euW) {
         euW.getTemperature();
         euW.showTemperature();
    }
}
```

The client uses the EUWeatherStation methods

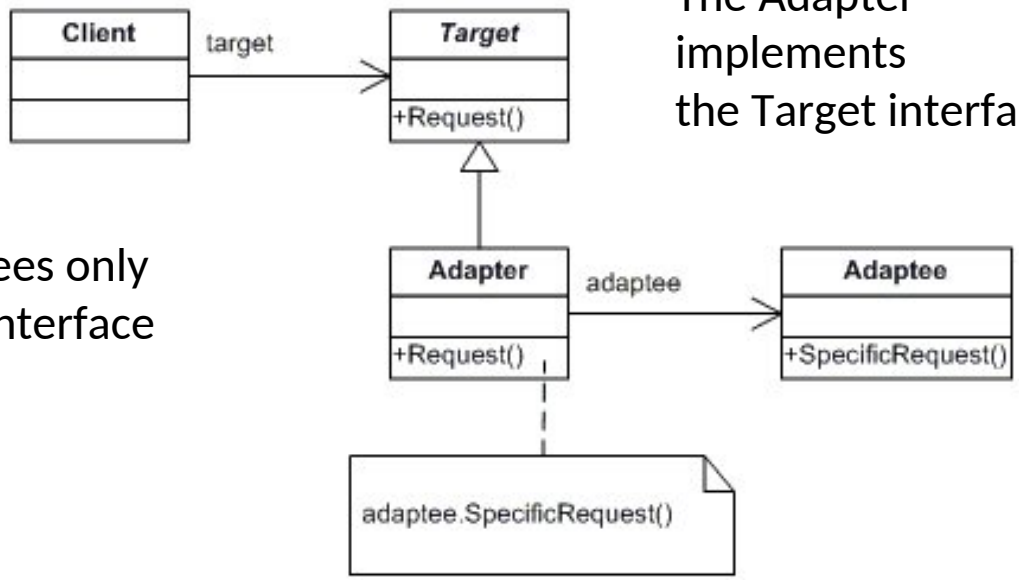
Roles

Client	Adapter	Adaptee
The Client is implemented against the target interface	The Adapter implements the target interface and holds an instance of the Adaptee	

- The client makes a request to the adapter by using the target interface.
- The adapter translates the request into one or more calls on the adaptee using the adaptee interface.
- The client receives the results of the call and never knows there is an adapter doing the translation.

The Adapter Pattern defined

- The Adapter Pattern converts the interface of a class into an other interface the clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.



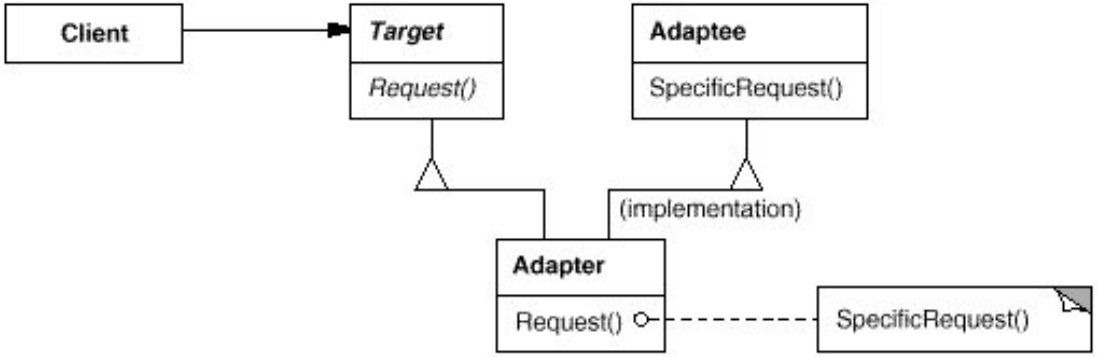
The Adapter implements the Target interface.

The client sees only the Target interface

All requests get delegated to the Adaptee.

Object and class adapters

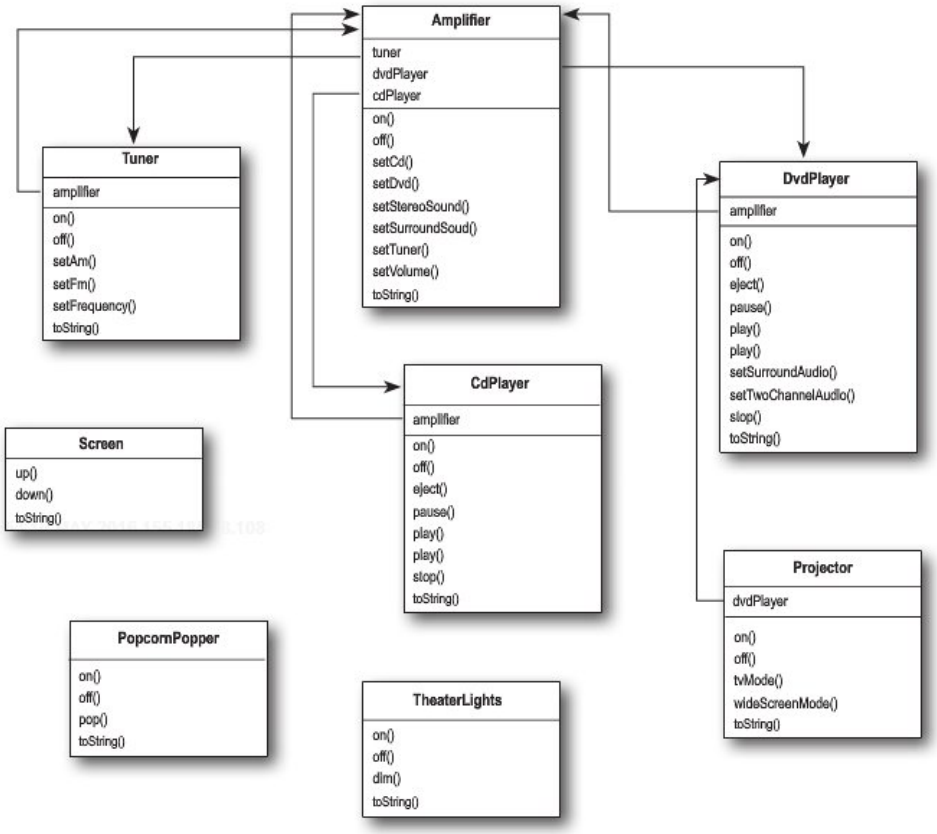
- There are two kinds of adapters: object adapters and class adapters.
- The class adapter



A class adapter uses multiple inheritance to adapt one interface to another … this is not possible in java!

The Facade Pattern

- A facade is an object that provides a simplified interface to a larger body of code, such as a class library.
- Let us suppose to have to manage a home theater.
  - If we want to watch a movie we have to (e.g.)
    - Dim the lights
    - Put the screen down
    - Turn the projector on
    - Put the projector on wide-screen mode
    - Turn the sound amplifier on
    - Set the amplifier to DVD input
    - Set the amplifier to surround sound
    - Set the amplifier volume to medium (5)
    - Turn the DVD player on
    - Start the DVD player playing12111098
    - Set the projector input to DVD



In terms of classes and methods

```
lights.dim(10);
screen.down();
projector.on();
projector.setInput(dvd);
projector.wideScreenMode();
amp.on();
amp.setDvd(dvd);
amp.setSurroundSound();
amp.setVolume(5);
dvd.on();
dvd.play(movie);
```

- This involves 5 classes!
- A similar situation happens if we want to turn everything off or to perform other operations!

Managing complexity

- With the Facade Pattern you can take a complex subsystem and make it easier to use by implementing a Facade class that provides a more reasonable interface.
- We can create a new class HomeTheaterFacade, which exposes a few simple methods
  - For example watchMovie().
- The Facade class treats the home theater components as a subsystem,
  - E.g., it calls on the subsystem to implement its watchMovie() method.
- The client code now calls methods on the home theater Facade, not on the subsystem.
  - To watch a movie we just call one method, watchMovie(), and it communicates with the rest of the classes
- The Facade still leaves the subsystem accessible to be used directly.
  - The functionality of the subsystem classes are still available

Implementing the simplified interface

```
public class HomeTheaterFacade {
    Amplifier amp;
    Tuner tuner;
    DvdPlayer dvd;
    CdPlayer cd;
    Projector projector;
    TheaterLights lights;
    Screen screen;
    public HomeTheaterFacade(Amplifier amp,
                Tuner tuner,
                DvdPlayer dvd,
                CdPlayer cd,
                Projector projector,
                Screen screen,
                TheaterLights lights) {
        this.amp = amp;
        this.tuner = tuner;
        this.dvd = dvd;
        this.cd = cd;
        this.projector = projector;
        this.screen = screen;
        this.lights = lights;
}
         // other methods here}
```

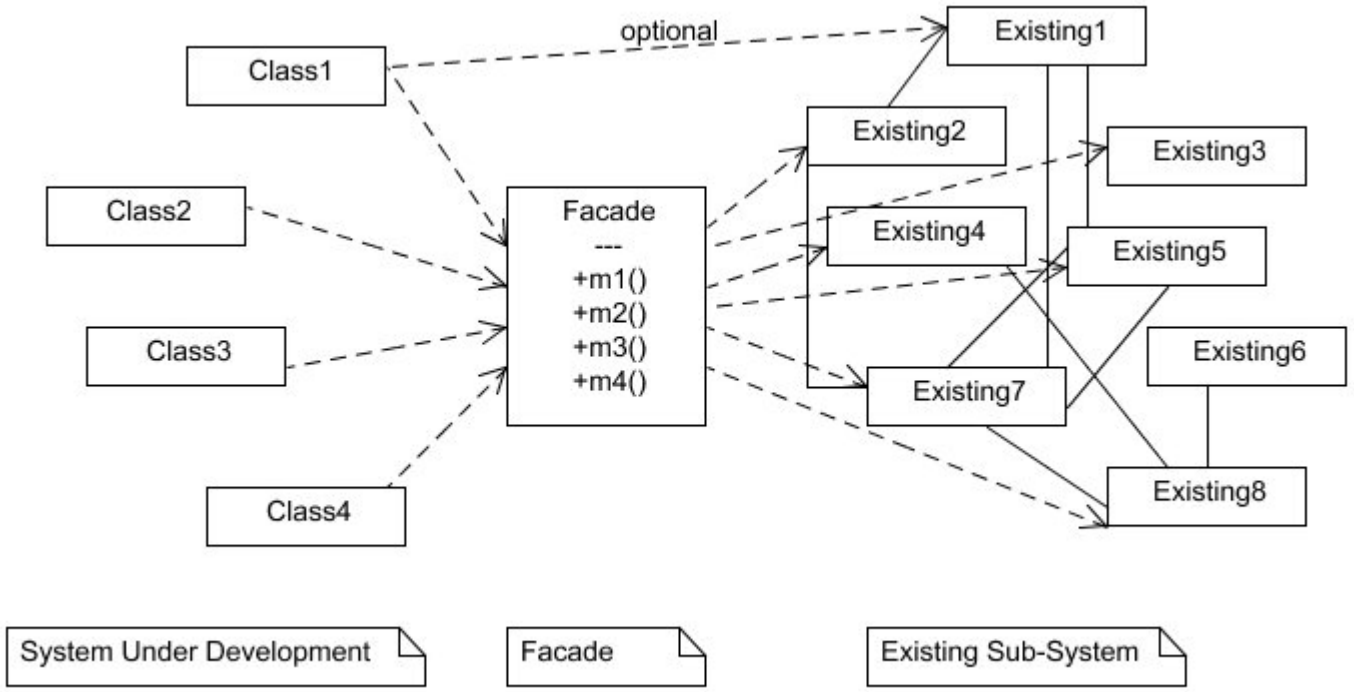
Implementing the simplified interface (2)

For instance one of the method could be:

```
Public void watchMovie(movie){
        lights.dim(10);
        screen.down();
        projector.on();
        projector.setInput(dvd);
        projector.wideScreenMode();
        amp.on();
        amp.setDvd(dvd);
        amp.setSurroundSound();
        amp.setVolume(5);
        dvd.on();
        dvd.play(movie);
        }
```

Facade Pattern defined

- The Facade Pattern provides a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.



The Principle of Least Knowledge (aka Principle of Demeter)

- Design Principle (Principle of Least Knowledge)
  Talk only to your immediate friends.
  - For any object, be careful of the number of classes it interacts with and also how it comes to interact with those classes.
  - This principle prevents us from creating designs that have a large number of classes coupled together so that changes in one part of the system cascade to other parts.
- From any method in that object, the principle tells us that we should only invoke methods that belong to:
  - The object itself
  - Objects passed in as a parameter to the method
  - Any object the method creates or instantiates
  - Any components of the object

Note: the guidelines tell us not to call methods on objects that are returned from calling other methods!

The Principle of Least Knowledge example

- How many classes is this code coupled to?

```
public float getTemp() {
      return station.getThermometer().getTemperature();
}
```

The Principle of Least Knowledge example

```
public float getTemp() {
    Thermometer thermometer = station.getThermometer();
    return thermometer.getTemperature();
}
```

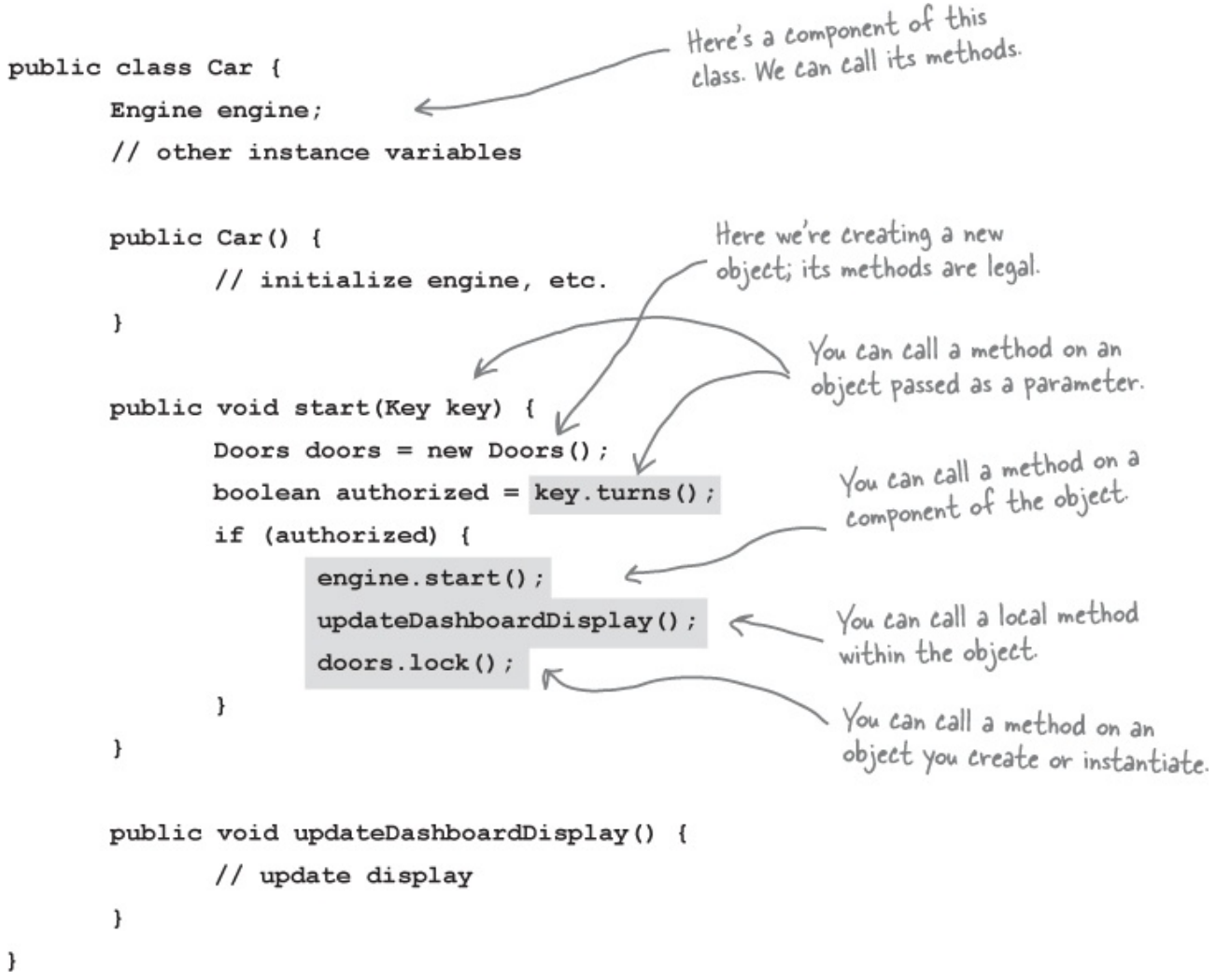
- Without the Principle
  - we get the thermometer object from the station and then call the getTemperature() method ourselves.

```
public float getTemp() {
    return station.getTemperature();
}
```

- With the Principle
  - We add a method to the Station class that makes the request to the thermometer for us.
    - This reduces the number of classes we're dependent on.

While the principle reduces the dependencies between objects and studies have shown this reduces software maintenance, applying this principle results in more "wrapper" classes being written to handle method calls to other components. This can result in increased complexity and development time as well as decreased runtime performance.

Applying the Least Knowledge Principle




```
public class Car {
        Engine engine;
        // other instance variables

        public Car() {
                // initialize engine, etc.
        }

        public void start(Key key) {
                Doors doors = new Doors();
                boolean authorized = key.turns();
                if (authorized) {
                        engine.start();
                        updateDashboardDisplay();
                        doors.lock();
                }
        }

        public void updateDashboardDisplay() {
                // update display
        }
}
```

Example

- Does this class violate the Least Knowledge Principle?

```
public House {
     WeatherStation station;

    // other methods and constructor

    public float getTemp() {
         Thermometer thermometer = station.getThermometer();
         return getTempHelper(thermometer);
    }

    public float getTempHelper(Thermometer thermometer) {
        return thermometer.getTemperature();
    }
}
```

Key Points

- ÉWhen you need to use an existing class and its interface is not the one you need, use an adapter.
  - An adapter changes an interface into one a client expects.
  - The effort for implementing an adapter depends on the target interface.
  - There are two forms of the Adapter Pattern: object and class adapters. Class adapters require multiple inheritance.
- ÉWhen you need to simplify and unify a large interface or complex set of interfaces, use a facade.
  - A facade decouples a client from a complex subsystem.
  - Implementing a façade requires that we compose the facade with its subsystem and use delegation to perform the work of the facade.
  - You can implement more than one facade for a subsystem.
- An adapter wraps an object to change its interface, a decorator wraps an object to add new behaviors and responsibilities, and a facade "wraps" a set of objects to simplify.

The Template Method Pattern

- Let us consider two classes for supporting the preparation of tea and (USA) coffee

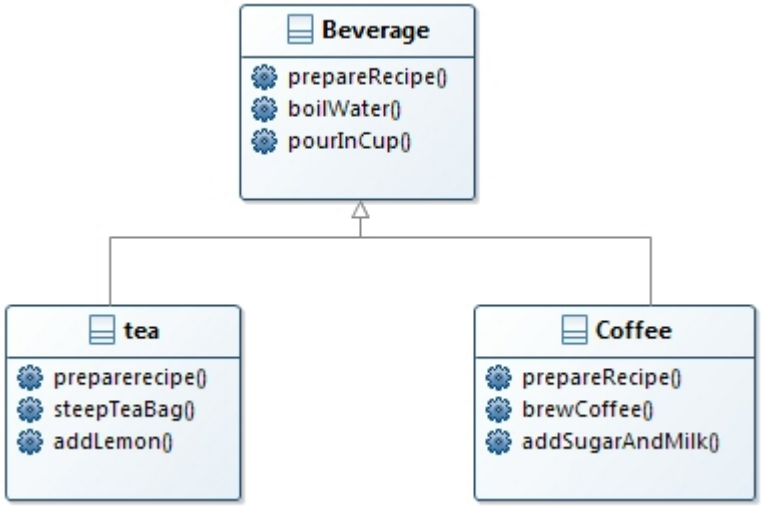
```
public class Tea {
    void prepareRecipe() {
        boilWater();
        steepTeaBag();
        pourInCup();
        addLemon();
    }
    public void boilWater() {
        System.out.println("Boiling ");
    }
    public void steepTeaBag() {
        System.out.println("Steeping the
tea");
    }
 public void addLemon() {
        System.out.println("Adding
Lemon");
    }
 public void pourInCup() {
        System.out.println("Pouring into
cup");
    }
}
```

```
public class Coffee {
     void prepareRecipe() {
        boilWater();
        brewCoffeeGrinds();
        pourInCup();
        addSugarAndMilk();
    }
    public void boilWater() {
        System.out.println("Boiling");
    }
    public void brewCoffeeGrinds() {
        System.out.println("Dripping
Coffee through filter");
    }
    public void pourInCup() {
        System.out.println("Pouring into
cup");
    }
    public void addSugarAndMilk() {
        System.out.println("Adding Sugar
and Milk");
    }
}
```

The Template Method Pattern

- The code is duplicated …
  - How can we design the classes reduce the duplications?

Abstracting the process



- The prepareRecipe() method differs in each subclass, so it is defined as abstract.
- Each subclass overrides the prepareRecipe() method and implements its own recipe.
- The boilWater() and pourInCup() methods are shared by both subclasses, so they are defined in the superclass.

Is a better abstraction possible?

Restructuring the model

- The methods which differ in the subclasses can be unified
  - addLemon(), addMilkAndSugar(), -- > addCondiments()
  - stepteaBag(), brewCoffee() -- > brew()
- and defined as abstract in the superclass
  - They will be implemented in the specific subclasses Tea and Coffee classes
- The prepareRecipe() method
  - is the same in both the classes
  - then can be made final in the superclass

Meet the Template Method

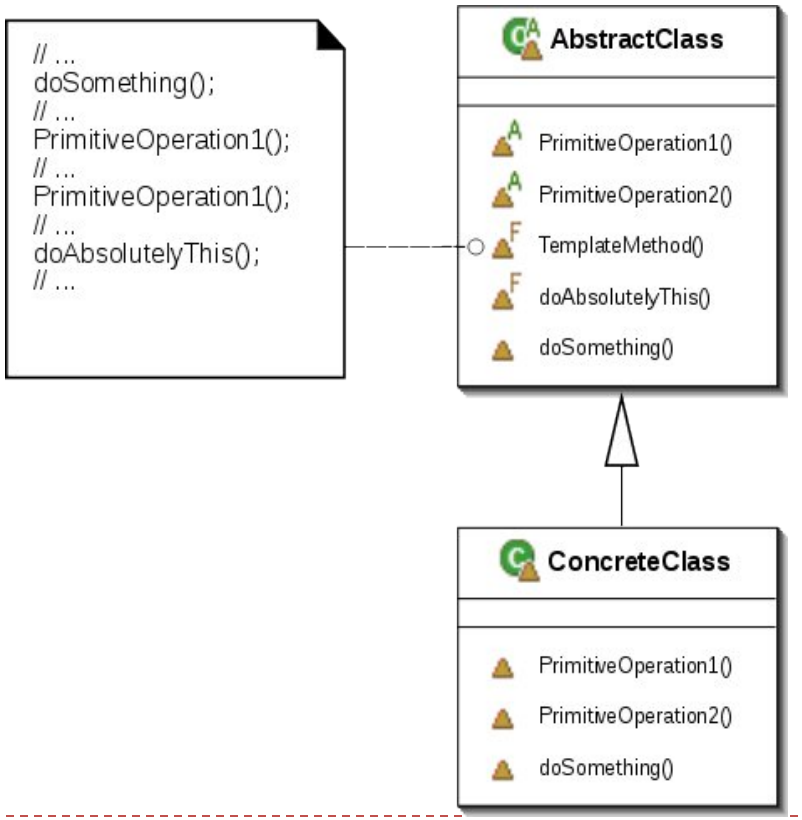
```
public abstract class CaffeineBeverage {
   void final prepareRecipe() {
         boilWater();
         brew();
         pourInCup();
         addCondiments();
                  }
   abstract void brew();
   abstract void addCondiments();

   void boilWater() {
        // implementation
   }
   void pourInCup() {
        // implementation
   }
}
```

- The Template Method defines the steps of an algorithm and allows subclasses to provide the implementation for one or more steps.

Template Method Pattern defined

- The Template Method Pattern defines the skeleton of an algorithm in a method, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.



The AbstractClass contains the template method ...and abstract versions of the operations used in the template method.

Another possible implementation including a hook()

```
abstract class AbstractClass {

    final void templateMethod() {
        primitiveOperation1();
        primitiveOperation2();
        concreteOperation();
        hook();
    }
    abstract void primitiveOperation1();

    abstract void primitiveOperation2();

    final void concreteOperation() {
        // implementation here
    }
    void hook() {}
}
```

We can also have concrete methods that do nothing by default; we call these “hooks”.

Subclasses are free to override these but don’t have to.

hook()

- A hook is a method that is declared in the abstract class, but only given an empty or default implementation.
  - This gives subclasses the ability to “hook into” the algorithm at various points, if they wish;
  - a subclass is also free to ignore the hook.

```
public abstract class BeverageWithHook {

    final void prepareRecipe() {
        boilWater();
        brew();
        pourInCup();
        if (wantsCondiments()) {
            addCondiments();
        }
    }
    abstract void brew();
    abstract void addCondiments();
    void boilWater() {
        System.out.println("Boiling water");
    }
    void pourInCup() {
        System.out.println("Pouring
into cup");
    }

    boolean wantsCondiments() {
        return true;
    }
}
```

This is a hook!

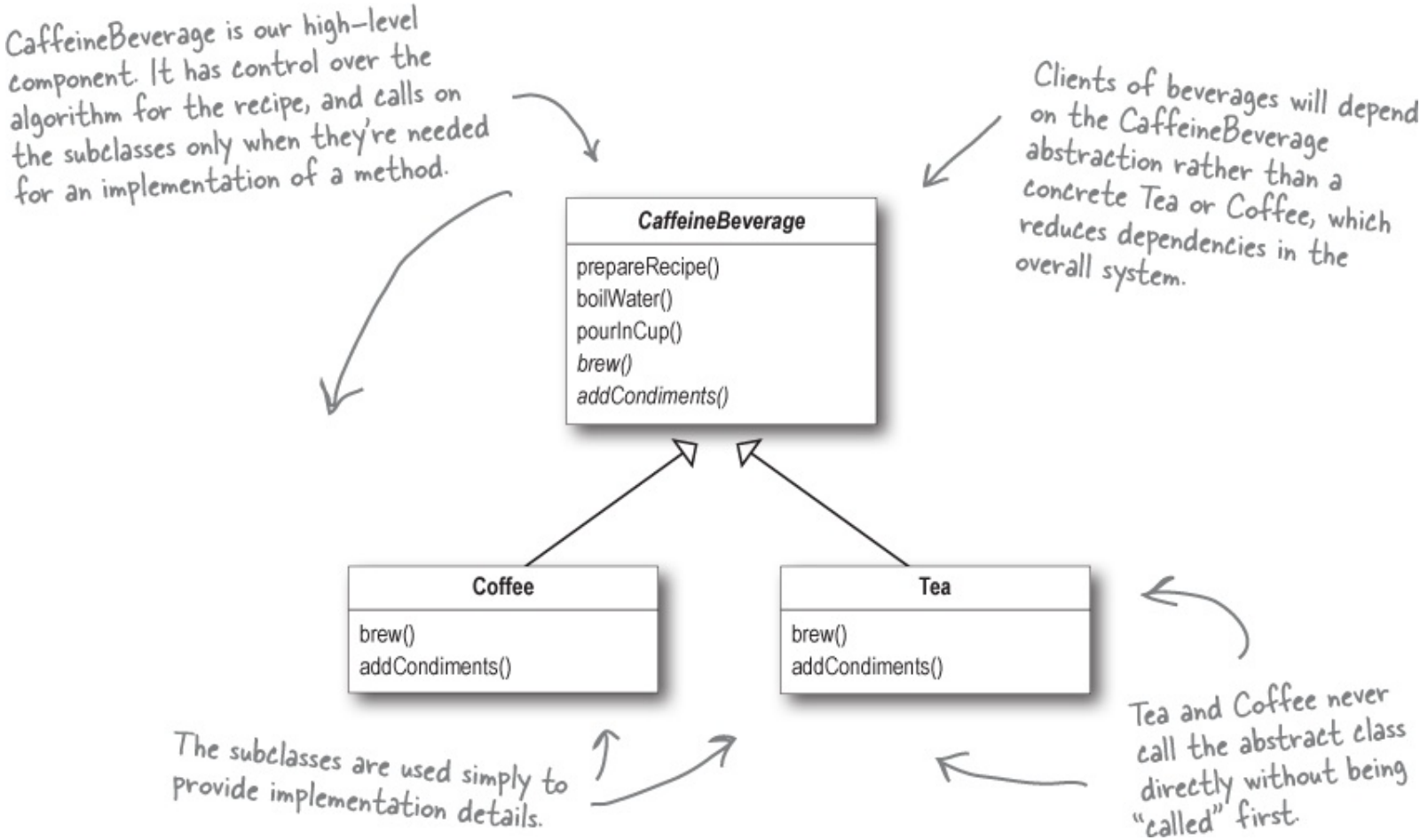
Using the hook

```
public class CoffeeWithHook extends BeverageWithHook {
…
    public boolean customerWantsCondiments() {
        String answer = getUserInput();
        if (answer.toLowerCase().startsWith("y")) {
            return true;
        } else {
            return false;
        }
    }
    private String getUserInput() {
      String answer = null;
      System.out.print("Would you like milk with your coffee (y/n)? ");
      BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
      try {
            answer = in.readLine();
      } catch (IOException ioe) {
            System.err.println("IO error trying to read your answer");
      }
      if (answer == null) {
            return "no";
      }
      return answer;
    }
}
```

The Hollywood Principle

- Design Principle(The Hollywood Principle)
  Don't call us, we'll call you.
- The Hollywood Principle gives us a way to prevent "dependency rot."
  - You have high-level components depending on low-level components depending on high-level components depending on sideways …
  - When rot sets in, no one can easily understand the way a system is designed.
- With the Hollywood Principle, we allow low-level components to hook themselves into a system, but the high-level components determine when they are needed, and how.
  - The high-level components give the low-level components a "don't call us, we'll call you" treatment.

The Hollywood Principle applied to the Template Method



Sorting with Template Method

- The designers of the Java Arrays class have provided us with a handy template method for sorting.

```
public static void sort(Object[] a) {
    Object aux[] = (Object[])a.clone();
    mergeSort(aux, a, 0, a.length, 0);
}

private static void mergeSort(Object src[], Object dest[],
  int low, int high, int off)
{
    // a lot of other code here
    for (int i=low; i<high; i++){
        for (int j=i; j>low &&
             ((Comparable)dest[j-1]).compareTo((Comparable)dest[j])>0;
j--)
        {
            swap(dest, j, j-1);
        }
    }
    // and a lot of other code here
}
```

The mergeSort() method contains the sort algorithm, and relies on an implementation of the compareTo() method to complete the algorithm.

What is compareTo()?

- The compareTo() method compares two objects and returns whether one is less than, greater than, or equal to the other.
  - sort() uses this as the basis of its comparison of objects in the array.
- The problem here is that java.Arrays is not subclass of anything
  - It is not possible to apply the template method as introduced
- Java designers made use of the Comparable interface.
  - All you have to do is implement this interface, which has one method: compareTo().

Comparing people

```
public class People implements Comparable {
    String name;
    int weight;

    public People(String name, int weight) {
        this.name = name;
        this.weight = weight;
    }

    public int compareTo(Object object) {

        People person = (People)object;

        if (this.weight < person.weight) {
            return -1;
        } else if (this.weight == person.weight) {
            return 0;
        } else { // this.weight > person.weight
            return 1;
        }
    }
}
```

Applets

- Concrete applets make extensive use of hooks to supply their own behaviors.

```
public class MyApplet extends Applet {
    String message;

    public void init() {
        message = "Hello World, I'm alive!";
        repaint();
    }

    public void start() {
        message = "Now I'm starting up...";
        repaint();
    }

    public void stop() {
        message = "Oh, now I'm being stopped...";
        repaint();
    }

    public void paint(Graphics g) {
        g.drawString(message, 5, 15);
    }
}
```

All methods are hooks that allow the customization of the Applet

The template method key points

- A “template method” defines the steps of an algorithm, deferring to subclasses for the implementation of those steps.
- ÉThe Template Method Pattern gives us an important technique for code reuse.
  - The template method’s abstract class may define concrete methods, abstract methods, and hooks.
  - To prevent subclasses from changing the algorithm in the template method, declare the template method as final.
- The Hollywood Principle guides us to put decision making in high-level modules that can decide how and when to call low-level modules.
- The Strategy and Template Method Patterns both encapsulate algorithms, one by inheritance and one by composition.

References

- Eric Freeman, Elisabeth Robson, Bert Bates, Kathy SierraHead First Design Patterns: Building Extensible and Maintainable Object-Oriented Software, O'Reilly Media 2020
- Erich Gamma, Ralph Johnson, Richard Helm, John Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.